Data Structures & Algorithms Lab-04

Queue using Singly Linked List

Item	Details
Duration	150 Minutes
Programming Language	Java
Level	Intermediate
Topic	Queue Implementation using Singly Linked List

1. Lab Objectives

- Explain Queue and FIFO.
- Implement a queue using a singly linked list.
- Create a Node class.
- Implement enqueue, dequeue, peek, isEmpty, and display.
- Handle underflow and the single-element case.
- Develop a menu-driven program.
- Analyze operation complexity.

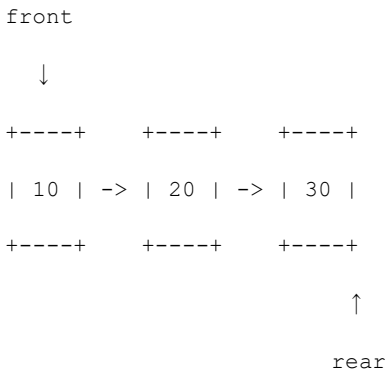
2. Lab Duration Breakdown

Activity	Time
Introduction and theory	15 min
Queue and FIFO	15 min
Singly linked list review	15 min
Node and Queue implementation	25 min
Basic Queue exercise	20 min
Queue operations	20 min
Menu-driven program	30 min
Testing and debugging	10 min
Total	150 min

3. Introduction to Queue

A Queue is a linear data structure in which insertion occurs at the rear and deletion occurs at the front. It follows FIFO (First In, First Out).

Examples include printer jobs, customer-service lines, CPU scheduling, ticket queues, and network packet processing.



4. Queue Terminology

Front: Points to the first node; deletion occurs here.

Rear: Points to the last node; insertion occurs here.

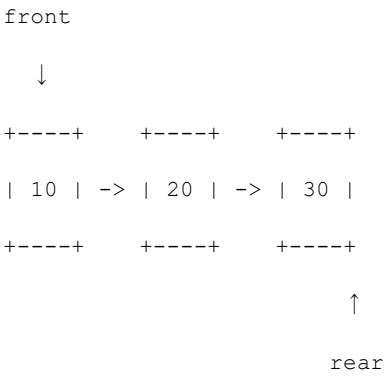
Underflow: Attempting to remove an element from an empty queue.

5. Basic Queue Operations

Operation	Description
enqueue()	Insert an element at the rear.
dequeue()	Remove an element from the front.
peek()	View the front element without removing it.
isEmpty()	Check whether the queue is empty.
display()	Display all elements.

6. Queue Using a Singly Linked List

Each node contains data and a next reference. The Queue maintains front and rear references.



7. Node Class

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```

Exercise 1 — Implement isEmpty()

Return true when the queue is empty.

```
boolean isEmpty() {
    return front == null;
}
```

Exercise 2 — Implement enqueue()

1. Create a new node.
2. If empty, set front and rear to the new node.
3. Otherwise connect the new node after rear.
4. Move rear to the new node.

```
void enqueue(int value) {
    Node newNode = new Node(value);

    if (isEmpty()) {
        front = newNode;
        rear = newNode;
    } else {
        rear.next = newNode;
        rear = newNode;
    }
}
```

Exercise 3 — Implement dequeue()

```
int dequeue() {
    if (isEmpty()) {
        System.out.println("Queue Underflow!");
        return -1;
    }

    int value = front.data;
    front = front.next;

    if (front == null) {
        rear = null;
    }

    return value;
}
```

Exercise 4 — Implement peek()

```
int peek() {
    if (isEmpty()) {
        System.out.println("Queue is empty!");
        return -1;
    }
    return front.data;
}
```

Exercise 5 — Implement display()

```
void display() {
    if (isEmpty()) {
        System.out.println("Queue is empty!");
        return;
    }

    Node temp = front;
    System.out.print("Front -> ");

    while (temp != null) {
        System.out.print(temp.data + " -> ");
        temp = temp.next;
    }

    System.out.println("NULL");
}
```

8. Complete Queue Class

```
class Queue {
    Node front;
    Node rear;

    boolean isEmpty() {
        return front == null;
    }

    void enqueue(int value) {
        Node newNode = new Node(value);
        if (isEmpty()) {
            front = rear = newNode;
        } else {
            rear.next = newNode;
            rear = newNode;
        }
    }

    int dequeue() {
        if (isEmpty()) {
            System.out.println("Queue Underflow!");
            return -1;
        }
        int value = front.data;
        front = front.next;
        if (front == null) rear = null;
        return value;
    }

    int peek() {
        if (isEmpty()) {
            System.out.println("Queue is empty!");
            return -1;
        }
        return front.data;
    }

    void display() {
        if (isEmpty()) {
            System.out.println("Queue is empty!");
            return;
        }
        Node temp = front;
        System.out.print("Front -> ");
        while (temp != null) {
            System.out.print(temp.data + " -> ");
            temp = temp.next;
        }
        System.out.println("NULL");
    }
}
```

9. Testing the Queue

```
public class Main {
    public static void main(String[] args) {
        Queue queue = new Queue();

        queue.enqueue(10);
```

```
        queue.enqueue(20);
        queue.enqueue(30);
        queue.enqueue(40);

        queue.display();
        System.out.println("Front element: " + queue.peek());
        System.out.println("Removed: " + queue.dequeue());

        queue.display();
        System.out.println("Removed: " + queue.dequeue());
        queue.display();
    }
}
```

Expected output:

```
Queue: 10 20 30 40
Front element: 10
Removed: 10
Queue: 20 30 40
Removed: 20
Queue: 30 40
```

10. Menu-Driven Queue

```
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Check Empty
6. Exit
```

Student task: integrate the operations into a do-while loop using Scanner and a switch statement.

11. Program Trace

Operation	Queue State	Explanation
enqueue(10)	10	First node; front and rear point to it.
enqueue(20)	10 -> 20	20 becomes rear.
enqueue(30)	10 -> 20 -> 30	30 becomes rear.
dequeue()	20 -> 30	10 is removed.
enqueue(40)	20 -> 30 -> 40	40 becomes rear.
dequeue()	30 -> 40	20 is removed.

12. Important Edge Cases

- Dequeue from an empty queue → Queue Underflow.
- Peek from an empty queue → Queue is empty.
- After deleting the only node, front and rear must both become null.
- Repeated operations must preserve FIFO order.

13. Time Complexity

Operation	Complexity

enqueue()	O(1)
dequeue()	O(1)
peek()	O(1)
isEmpty()	O(1)
display()	O(n)

14. Student Tasks

- **size() —** Return the number of elements in the queue.
- **search(int value) —** Search for a value.
- **Count Even/Odd —** Count even and odd values.

15. Viva Questions

5. What is a queue?
6. What does FIFO mean?
7. What is the difference between front and rear?
8. Where is insertion performed?
9. Where is deletion performed?
10. What is queue underflow?
11. Why maintain a rear pointer?
12. What happens when the last element is removed?
13. What is the complexity of enqueue()?
14. Why can a linked-list queue grow dynamically?
15. What happens when front == null?
16. What is the difference between a stack and a queue?

17. Final Lab Task

Develop a complete Queue Management System using a singly linked list with:

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Is Empty
6. Size
7. Search
8. Count Even/Odd
9. Exit

Submission Checklist

- Java source code